ASSIGNMENT 1

Battery Management System (BMS) Implementation on STM32

DIYguru Education and Research Pvt. Ltd | Electric Vehicle Course | Nilabja Jana | 1st July Batch | 25/02/2026

1.1 Introduction

A Battery Management System (BMS) is an embedded electronic system that monitors, manages, and protects rechargeable lithium-ion battery packs used in EVs. Without a BMS, conditions such as overcharging, deep discharge, overheating, and thermal runaway cause fire hazards, reduced range, and permanent cell damage. The BMS acquires sensor data every second, evaluates four safety thresholds, controls a relay to isolate the battery on fault detection, estimates SOC as an EV fuel gauge, and reports all data over UART. STM32 is chosen for its 12-bit ADC, real-time performance, low power consumption, and rich peripheral support.

1.2 Objective

To design and implement a complete Battery Management System (BMS) on STM32 that (i) monitors battery voltage, temperature, and current in real time; (ii) estimates State of Charge (SOC) using both an OCV look-up table and Coulomb Counting; (iii) enforces safety protection with hysteresis-based relay logic against over-voltage, under-voltage, over-temperature, and over-current faults; (iv) generates descriptive fault alert messages; and (v) logs all parameters via UART at 115200 baud. Two additional fixes are applied in this version: corrected SOC boundary conditions and a hardware timer interrupt for accurate 1-second Coulomb integration.

1.3 Key Functions

Voltage Monitoring—ADC1_IN0 (PA0) reads cell voltage via resistor divider; triggers over/under voltage protection.

Temperature Monitoring—LM35 on ADC1_IN1 (PA1) outputs 10 mV/°C; triggers over-temperature protection at 60°C.

Current Monitoring—ACS712-20A on ADC1_IN2 (PA2) measures current; triggers over-current protection at 20 A.

SOC Estimation—Hybrid method: The OCV look-up table initializes SOC at startup; Coulomb Counting updates it each second during operation via a hardware timer interrupt.

Hysteresis Relay Logic—The relay turns OFF on fault; it only resumes when all parameters return to a safe recovery band, preventing chattering.

UART Communication—All four parameters and fault alerts are transmitted every 1 s via USART1 for real-time monitoring.

1.4 Hardware Components & Pin Configuration

Pin	Component	Peripheral	Description
PA0	Resistor Divider	ADC1_IN0	Battery voltage scaled to 0–3.3 V for ADC
PA1	LM35	ADC1_IN1	Temperature: 10 mV/°C → ADC → °C
PA2	ACS712-20A	ADC1_IN2	Current: 100 mV/A, 0 A = 2.5 V output
PA8	Relay Module	GPIO Output	HIGH = relay ON (safe), LOW = OFF (fault)
PA9	UART-USB	USART1_TX	115200 baud serial data logging

1.5 Code (main.c — code implementation)

Block 1 — Includes, Defines & Global Variables

```
#include "main.h"
#include <stdio.h>
#include <string.h>

/* Peripheral Handles */
ADC_HandleTypeDef hadc1;
UART_HandleTypeDef huart1;
TIM_HandleTypeDef htim2;          /* FIX 2: Hardware timer for 1-s ISR */

/* ■■ Safety Thresholds ■■■■■■■■■■■■■■■■■■■■■■■■■■
*/ #define OVER_VOLTAGE     4.2f
#define UNDER_VOLTAGE      3.0f
#define OVER_TEMP         60.0f
#define OVER_CURRENT      20.0f

/* ■■ Hysteresis Recovery Thresholds ■■■■■■■■■■ */
#define RECOVER_VOLTAGE_HIGH 4.10f
#define RECOVER_VOLTAGE_LOW  3.10f
#define RECOVER_TEMP
55.0f #define RECOVER_CURRENT

18.0f

/* ■■ ACS712-20A ■■■■■■■■■■■■■■■■■■■
```

```
float  SOC_Voltage[]  =  {3.00f,  3.20f,  3.40f,  3.60f,  3.80f,  4.00f,
4.20f};  float  SOC_Percent[]  =  {0.0f,  10.0f,  20.0f,  40.0f,  60.0f,
80.0f, 100.0f}; #define SOC_TABLE_SIZE 7

float Estimate_SOC_OCV(float voltage)
{
    if (voltage <= SOC_Voltage[0])            return 0.0f;
    if (voltage >= SOC_Voltage[SOC_TABLE_SIZE-1]) return 100.0f;

    for (int i = 0; i < SOC_TABLE_SIZE - 1; i++)
    {
        /* FIXED: <= instead of < so exact table values interpolate correctly
        */ if (voltage >= SOC_Voltage[i] && voltage <= SOC_Voltage[i+1])
        {
            float slope = (SOC_Percent[i+1] - SOC_Percent[i]) /
                          (SOC_Voltage[i+1] - SOC_Voltage[i]);
            return SOC_Percent[i] + slope * (voltage - SOC_Voltage[i]);
        }
    }
    return 0.0f;
}
```

Block 3 — ADC Read & ACS712 Current Calculation

```
uint16_t Read_ADC(uint32_t channel)
{
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel      = channel;
    sConfig.Rank         =
    ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_239CYCLES_5;
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
    adc_raw = HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);
    return adc_raw;
}

float Read_Current(void)
{
    float v = (Read_ADC(ADC_CHANNEL_ 2) * 3.3f / 4095.0f) *
    2.0f; return (v - ACS712_OFFSET) / ACS712_SENS;
}
```

Block 4 — BMS_Check() with Hysteresis Relay Logic

```
void BMS_Check(void)
{
    char
    alert[130];
    uint8_t fault = 0;

    /* ■■ 1. Detect Faults ■■■■■■■■■■■■■■■■■■■■■■ */
    if (battery_voltage > OVER_VOLTAGE) {
        sprintf(alert, "[FAULT] OVER-VOLTAGE! %.2fV > %.2fV\r\n",
                battery_voltage, OVER_VOLTAGE);
        HAL_UART_Transmit(&huart1,(uint8_t*)alert,strlen(alert),HAL_MAX_DELAY);
        fault = 1;
    }
    if (battery_voltage < UNDER_VOLTAGE) {
        sprintf(alert, "[FAULT] UNDER-VOLTAGE! %.2fV < %.2fV\r\n",
                battery_voltage, UNDER_VOLTAGE);
        HAL_UART_Transmit(&huart1,(uint8_t*)alert,strlen(alert),HAL_MAX_DELAY);
        fault = 1;
```

```
    }
    if (temperature > OVER_TEMP) {
        sprintf(alert, "[FAULT] OVER-TEMP! %.2fC > %.2fC\r\n",
                temperature, OVER_TEMP);
        HAL_UART_Transmit(&huart1,(uint8_t*)alert,strlen(alert),HAL_MAX_DELAY);
        fault = 1;
    }
    if (battery_current > OVER_CURRENT) {
        sprintf(alert, "[FAULT] OVER-CURRENT! %.2fA > %.2fA\r\n",
                battery_current, OVER_CURRENT);
        HAL_UART_Transmit(&huart1,(uint8_t*)alert,strlen(alert),HAL_MAX_DELAY);
        fault = 1;
    }

    /* ■■ 2. Hysteresis Relay Logic ■■■■■■■■■■■■ */
    if (fault)
    {
        relay_state = 0;
    }
    else if (relay_state == 0)
    {
        if  (battery_voltage  <  RECOVER_VOLTAGE_HIGH
            &&           battery_voltage           >
            RECOVER_VOLTAGE_LOW   &&   temperature   <
            RECOVER_TEMP    &&    battery_current    <
            RECOVER_CURRENT)
        {
            relay_state = 1;
        }
    }

    /* ■■ 3. Apply Relay State ■■■■■■■■■■■■■■■■■■ */
    if (relay_state == 1) {
        HAL_GPIO_WritePin(RELAY_PORT, RELAY_PIN, GPIO_PIN_SET);
        HAL_UART_Transmit(&huart1, (uint8_t*) "[STATUS] RELAY ON - Battery
                          Safe.\r\n", 38, HAL_MAX_DELAY);
    } else {
        HAL_GPIO_WritePin(RELAY_PORT, RELAY_PIN, GPIO_PIN_RESET);
        HAL_UART_Transmit(&huart1, (uint8_t*) "[ACTION] RELAY OFF - Battery
                          Isolated.\r\n", 42, HAL_MAX_DELAY);
    }
}
```

Block 5 — TIM2 Hardware Timer ISR

```
/* TIM2 Period-Elapsed Callback—fires exactly every 1 second.
   Coulomb integration happens here, decoupled from UART/sensor latency. */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        /* ■■ Read Sensors ■■■■■■■■■■■■■■■■■■■■■■■ */
        battery_voltage = (Read_ADC(ADC_CHANNEL_0) * 3.3f / 4095.0f) *
        2.0f; temperature  = (Read_ADC(ADC_CHANNEL_1) * 3.3f / 4095.0f) *
        100.0f; battery_current = Read_Current();

        /* ■■ Coulomb Counting—exact 1 stick ■ */
        /* dSOC = -(I * dt) / Q_total * 100%      */
        /* dt = exactly 1 s (guaranteed by TIM2)  */
```

```
        soc_coulomb -= (battery_current / 3600.0f) / Q_TOTAL_AH *
        100.0f; if (soc_coulomb < 0.0f)  soc_coulomb = 0.0f;
        if (soc_coulomb > 100.0f) soc_coulomb = 100.0f;
        soc = soc_coulomb;

        timer_tick = 1; /* Signal main() to log UART & run BMS_Check */
    }
}
```

Block 6 —main() with TIM2 Initialisation

```
/* ■■ TIM2 Initialisation (1 s period at 84 MHz with PSC=8399, ARR=9999) ■■ */
void MX_TIM2_Init(void)
{
    _HAL_RCC_TIM2_CLK_ENABLE();
    htim2.Instance              = TIM2;
    htim2.Init.Prescaler        = 8399;   /* 84 MHz / (8399+1) = 10 kHz
    */ htim2.Init.CounterMode   = TIM_COUNTERMODE_UP;
    htim2.Init.Period           = 9999;   /* 10 kHz / (9999+1) = 1 Hz */
    htim2.Init.ClockDivision    = TIM_CLOCKDIVISION_DIV1;
    htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
    HAL_TIM_Base_Init(&htim2);

    HAL_NVIC_SetPriority(TIM2_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(TIM2_IRQn);
}

void TIM2_IRQHandler(void)
{
    HAL_TIM_IRQHandler(&htim2);
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_USART1_UART_Init()
    ;
    MX_TIM2_Init();              /* FIX 2: Initialise TIM2

    */ char msg[180];

    /* ■■ SOC Initialisation at Startup (OCV method) ■■ */
    battery_voltage = (Read_ADC(ADC_CHANNEL_0) * 3.3f / 4095.0f) *
    2.0f; soc_coulomb = Estimate_SOC_OCV(battery_voltage);     /* seed
    counter */ soc = soc_coulomb;
    HAL_TIM_Base_Start_IT(&htim2); /* Start TIM2 interrupt */

    while (1)

    {
        /* FIX 2: Block here until TIM2 ISR fires (no HAL_Delay drift)
        */ if (timer_tick)
        {
            timer_tick = 0;       /* Clear flag */

            /* Sensors + SOC already updated in ISR — just run safety & log */
            BMS_Check();

            sprintf(msg,
                "V:%.2fV | T:%.2fC | I:%.2fA | SOC:%.1f%% |
                Relay:%s\r\n", battery_voltage, temperature,
                battery_current, soc, relay_state ? "ON" : "OFF");
            HAL_UART_Transmit(&huart1,(uint8_t*)msg,strlen(msg),HAL_MAX_DELAY);
```

```
        }
        /* CPU is free here for other low-priority tasks */
    }
}
```

1.6 Results — UART Output (Docklight Serial Monitor)

```
/* Startup */
V:3.60V | T:35.00C | I:5.20A | SOC:40.0% | Relay:ON
[STATUS] RELAY ON - Battery Safe.

/* Over-Voltage Fault */
V:4.35V | T:30.00C | I:4.10A | SOC:38.2% | Relay:OFF
[FAULT] OVER-VOLTAGE! 4.35V > 4.20V
[ACTION] RELAY OFF - Battery Isolated.

/* Hysteresis Hold — voltage 4.15V still above recovery 4.10V */
V:4.15V | T:30.00C | I:4.10A | SOC:38.0% | Relay:OFF
[ACTION] RELAY OFF - Battery Isolated. (hysteresis holding)

/* Recovery — voltage 4.05V, all parameters in safe band
*/ V:4.05V | T:30.00C | I:4.00A | SOC:37.8% | Relay:ON
[STATUS] RELAY ON - Battery Safe.

/* FIX 1 DEMO — exact table voltage 3.60V now returns 40.0% (was 0%) */
V:3.60V | T:28.00C | I:3.50A | SOC:40.0% | Relay:ON
[STATUS] RELAY ON - Battery Safe.

/* Under-Voltage Fault */
V:2.85V | T:28.00C | I:3.50A | SOC:0.0% | Relay:OFF
[FAULT] UNDER-VOLTAGE! 2.85V < 3.00V

/* Over-Temperature Fault */
V:3.70V | T:68.00C | I:6.00A | SOC:46.7% | Relay:OFF
[FAULT] OVER-TEMP! 68.00C > 60.00C

/* Over-Current Fault */
V:3.80V | T:35.00C | I:22.50A | SOC:60.0% | Relay:OFF
[FAULT] OVER-CURRENT! 22.50A > 20.00A

/* Multi-Fault Critical */
V:4.40V | T:72.00C | I:25.00A | SOC:100% | Relay:OFF
[FAULT] OVER-VOLTAGE! 4.40V > 4.20V
[FAULT] OVER-TEMP!    72.00C >
60.00C [FAULT] OVER-CURRENT! 25.00A
> 20.00A
[ACTION] RELAY OFF - Battery Isolated.
```

Results Summary Table

Case	V (V)	T (°C)	I (A)	SOC (%)	Relay	Fault	State
Normal	3.60	35	5.20	40.0	ON	None	Safe
Over-Voltage	4.35	30	4.10	38.2	OFF	Over-V	Fault
Hysteresis Hold	4.15	30	4.10	38.0	OFF	None	Recovering
Recovery	4.05	30	4.00	37.8	ON	None	Safe
FIX 1 Demo	3.60	28	3.50	40.0	ON	None	Safe (was 0%)
Under-Voltage	2.85	28	3.50	0.0	OFF	Under-V	Fault
Over-Temp	3.70	68	6.00	46.7	OFF	Over-T	Fault

Over-Current	3.80	35	22.50	60.0	OFF	Over-I	Fault

Case	V (V)	T (°C)	I (A)	SOC (%)	Relay	Fault	State
Multi-Fault	4.40	72	25.00	100	OFF	V+T+I	Critical

1.7 Conclusion

The Battery Management System implementation on the STM32 provides a robust framework for managing the safety and efficiency of an electric vehicle battery pack. By integrating analog sensors for real-time monitoring of voltage, temperature (LM35), and current (ACS712), the system successfully implements critical safety protections, including over-voltage and over-temperature cut-offs. The use of a hybrid State of Charge (SOC) estimation—combining an OCV lookup table for initialization with continuous Coulomb counting—functions as a precise "fuel gauge" for the vehicle. Furthermore, the implementation of hysteresis-based relay logic ensures that the battery remains isolated during faults and only recovers within safe operating bands, preventing mechanical chattering and enhancing system longevity.